Automatisation du développement

Points de la semaine

- Tester son code ?!
- oui ! et de maniere automatique :
- Tester avec PHPUnit
- Tester avec Vitest

Tester son code ✅

- Pourquoi faire des tests ?
- Comment écrire ses tests ?

Pourquoi

- Vérifier les fonctionnalités
- Prévenir l’apparition de bugs
- Faciliter les modifications / améliorations
- Eviter les régressions

Différents tests

- Tests unitaires
- Tests d’intégration
- Tests fonctionnels
- Tests de performance et de charge
- Aussi : de bout en bout, d'acceptance, smoke test, etc.

Tests unitaires

- très bas niveau
- tester juste une fonction simple, ou un composant individuel
- automatisables et executables rapidement par la CI

Tests d’intégration

- vérifient la coopération entre plusieurs briques
- ex : tester l'interaction avec la BDD
- plus longs et couteux

Tests fonctionnels

- exigences métiers
- ex : tester que la BDD renvoie telle valeure

Tests de performance et de charge

- mesurent la fiabilité, la vitesse
- temps de réponse
- recherche de goulots d'étranglement

Autres tests :

- smoke tests : tests concentrés sur les parties critiques, très rapide et prioritaires
- tests d'acceptance : validation d'une spécification fonctionnelle
- tests de bout en bout : parcours client de A à Z

Comment

Les bonnes pratiques

- Objectifs clairs : Tester ce qui est testable (et ne pas tester si c'est inutile)
- Tester le plus tot possible
- Tester tous les cas de figures, meme les erreurs
- Automatiser les tests, et mesurer leur couverture
- Bien structurer ses tests

Le TDD

- Écrire les tests avant les fonctions / modifications
- Empeche la regression
- Clarifie le fonctionnement
- Améliore le code coverage

Mocking

- Remplace certaines parties du code lors des tests (ex. des objets, des DI, etc.)
- Simuler des comportements externes
- Contrôler les tests
- ⚠️ à ne pas trop mocker

Fonction de setup

- Code executé et variables initialisés avant les tests.
- Évite les répétitions
- `setUp()`, `beforeAll()`, `beforeEach()`, etc.

Le code coverage (couverture)

- Identifier les zones non testées
- Mesurer la qualité des tests
- Réduire les risques de bug

	Code Coverage								
	Lines			Functions and Methods			Classes and Traits		
Total		63.76%	227 / 356		53.77%	57 / 106		52.38%	33 / 63
Console		66.67%	2 / 3		50.00%	1 / 2		0.00%	0 / 1
Exceptions		100.00%	2 / 2		100.00%	1 / 1		100.00%	1 / 1
Http		75.24%	158 / 210		61.82%	34 / 55		76.00%	19 / 25
Listeners		0.00%	0 / 18		0.00%	0 / 3		0.00%	0 / 1
Mail		44.32%	39 / 88		35.29%	12 / 34		25.93%	7 / 27
Models		n/a	0 / 0		n/a	0 / 0		n/a	0 / 0
Providers		66.67%	18 / 27		77.78%	7 / 9		66.67%	4 / 6
Services		100.00%	8 / 8		100.00%	2 / 2		100.00%	2 / 2

Les outils

Backend : PHPUnit

https://phpunit.de/

- Librairie PHP
- Fonctions assertXXXX("attendu", "résultat)

Utilisation

- Fichier de configuration `phpunit.xml`
- Commande `./vendor/bin/phpunit`

Exemple

```php
// MathTest.php
use PHPUnit\Framework\TestCase;

class MathTest extends TestCase
{
    public function testSquare(): void
    {
        $math = new MyMathClass();
        $this->assertEquals(4, $math->square(2));
    }
}
```

```
PHPUnit 10.2.2 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.3.1
Configuration: /Users/arti-13/Projects/microservices/email-microservice/phpunit.xml

.............................................................  63 / 142 ( 44%)
.............................................................. 126 / 142 ( 88%)
................                                              142 / 142 (100%)

Time: 00:04.809, Memory: 40.00 MB

OK (142 tests, 208 assertions)
```

```
PHPUnit 10.5.7 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.3.1
Configuration: /Users/arti-13/Projects/Perso/IUT/devoir-3/phpunit.xml

F                                                                   1 / 1 (100%)

Time: 00:00.013, Memory: 8.00 MB

There was 1 failure:

1) App\Tests\ExempleTest::testExemple
Failed asserting that false is true.

/Users/arti-13/Projects/Perso/IUT/devoir-3/tests/ExempleTest.php:11

FAILURES!
Tests: 1, Assertions: 1, Failures: 1.
```

Interlude

- Dans le repo des exercices, répertoire « cours 3 INTERLUDES »
- Aller dans le dossier `phpunit`
- Lire le README
- Écrire les tests manquants
- Sortir un rapport code coverage

Frontend : Vitest

https://vitest.dev

- Package NPM, Fonctionne avec Vite
- Test des composants
- Test du comportement du navigateur
- Fichiers `spec.js` dans les dossiers `__tests__`

Utilisation

- Commande `vitest`
- Configuration :
  - `vitest.config.ts`
  - ou mode `test` dans `vite.config.ts`

Exemple

```
// sum.test.js
import { expect, test } from 'vitest'
import { sum } from './sum'

test('adds 1 + 2 to equal 3', () => {
  expect(sum(1, 2)).toBe(3)
})
```

```
 DEV  v0.28.4 /home/projects/vitejs-vite-b1avnh

 ✓ src/lib/__test__/Hello.test.ts (1)

 Test Files  1 passed (1)
      Tests  1 passed (1)
   Start at  13:56:00
   Duration  2.01s (transform 260ms, setup 112ms, collect 253ms, tests 13ms)


 PASS  Waiting for file changes...
       press h to show help, press q to quit
```

```
 DEV  v0.28.4 /home/projects/vitejs-vite-b1avnh

 ❯ src/lib/__test__/Hello.test.ts (1)
   ❯ Hello component (1)
     × should show message

──────────────────────────────────────────────────────────────────────────────── Failed Tests 1 ────────────────────────────────────────────────────────────────────────────────

 FAIL  src/lib/__test__/Hello.test.ts > Hello component > should show message
TestingLibraryElementError: Unable to find an element with the text: Hello. This could be because the text is broken
 up by multiple elements. In this case, you can provide a function for your text matcher to make your matcher more f
lexible.

Ignored nodes: comments, script, style
<body>
  <div>

  </div>
</body>
 ❯ Object.getElementError node_modules/@testing-library/dom/dist/config.js:37:19
 ❯ eval node_modules/@testing-library/dom/dist/query-helpers.js:76:38
 ❯ eval node_modules/@testing-library/dom/dist/query-helpers.js:52:17
 ❯ eval node_modules/@testing-library/dom/dist/query-helpers.js:95:19
 ❯ eval src/lib/__test__/Hello.test.ts:8:12
      6|     const { getByText } = render(StatusBar);
      7|
      8|     expect(getByText('Hello')).toBeInTheDocument();
       |            ^
      9|   });
     10| });

──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────[1/1]─

 Test Files  1 failed (1)
      Tests  1 failed (1)
   Start at  13:56:50
   Duration  2.06s (transform 267ms, setup 117ms, collect 247ms, tests 18ms)


 FAIL  Tests failed. Watching for file changes...
       press h to show help, press q to quit
```

Interlude

- Aller dans le dossier `vitest-cypress`
- Lire le README
- Écrire les tests Vitest
- Sortir un rapport code coverage

À votre tour 💪